Term Project: Movies Data Manager Using Data Structures (Fall 2025)

National University of Computer and Emerging Sciences (FAST-NUCES)
Department of Artificial Intelligence and Data Science

1. Project Overview

This term project requires students to design and implement a **Movies Data Manager** using C++ **without STL**. The purpose of this project is to assess the understanding of the students on the practical application of core data structures including **Stack Queue, BST/AVL Trees, Linked Lists, Hash Tables, and Graphs**. The system will manage a real-world dataset of movies extracted from the *IMDb 5000 Movie Dataset*, providing functionalities such as searching, recommendation, and relationship discovery among movies.

2. Dataset

Students are required to use the IMDb 5000 Movie Dataset available at the following link:

`https://www.kaggle.com/datasets/carolzhangdc/imdb-5000-movie-dataset`

The dataset contains approximately 5000 movies with multiple attributes such as *title, genre, director, actors, rating, duration, and year*. Students must parse and utilize relevant fields in their data structures.

3. Language and Restrictions

- **Programming Language:** C++
- **Restriction:** Use of STL (Standard Template Library) is strictly prohibited.

All data structures must be implemented manually. Violation will result in **zero marks**.

4. Project Duration and Group Policy

- **Project Deadline:** December 7th, 2025
- **Maximum Group Size:** 2 Members

Each group will present their project during the viva and demonstration sessions. Individual contribution will be evaluated; it is possible for one member to receive full marks while the other receives none based on performance.

5. System Overview

The system should read and organize data from the IMDb dataset using the required data structures. The **AVL Tree or BST** will act as the central storage for movies, while **Hash Tables and Linked Lists** will support fast access to actors, genres, and related information. **Graphs** will represent relationships between movies sharing similar attributes such as common actors or genres. Traversals on the graph will enable recommendation and path-finding features that simulate real-world recommender systems.

6. Required Data Structures

1. **AVL Tree / BST:** Store and organize movies based on their titles. Support insertion, deletion, and search operations.
2. **Hash Table:** Store actors or genres to allow fast access and mapping between movies and actors/genres.
3. **Linked List:** Maintain lists of genres, movies under a specific actor, or filmographies.
4. **Graph:** Represent relationships between movies that share actors or genres. Use graph traversal algorithms (BFS/DFS) for recommendations and path-finding.

7. Class Descriptions (Guidelines)

7.1. MovieNode Class

- Store information such as title, genre(s), actors, director, rating, and release year.
- Maintain references to linked lists of actors or genres.
- Provide methods for displaying and updating movie information.

7.2. ActorNode Class

- Store actor's name and maintain a linked list or tree of movies they have appeared in.
- Allow retrieval of co-actors and the list of movies in which the actor participated.

7.3. Graph Class

- Represent each movie as a vertex.
- Create edges between movies that share one or more attributes (e.g., actors or genres).
- Implement traversal algorithms (BFS/DFS) to recommend similar movies.
- Implement shortest path finding between two movies based on shared attributes.

7.4. HashTable and LinkedList Classes

- Use these structures to manage and index actor and genre information.
- Ensure efficient insertion and lookup operations.

8. Functional Requirements

1. **Dataset Import:** Parse the IMDb 5000 dataset file. For each movie, create appropriate nodes and insert them into the relevant data structures.
2. **Search Functionalities:** Search movies by title, actor, genre, year, or rating range.
3. **Graph Construction:** Each movie acts as a vertex. Edges are created between movies that share at least one actor or genre.
4. **Recommendation System:** Given a movie title, recommend other movies using graph traversal (BFS/DFS).
5. **Shortest Connection Path:** Implement functionality to find the shortest path between two movies, actors, or directors based on shared attributes.

9. Implementation Guidelines

- Use recursion wherever applicable, especially for traversing trees and graphs.
- Handle duplicate entries and invalid inputs gracefully.
- Use meaningful variable, class, and method names.
- Each method must include descriptive comments.

- Ensure proper indentation and formatting.
- Use dynamic memory allocation responsibly; avoid memory leaks.
- Global variables are discouraged unless necessary.
- Make assumptions where necessary with proper objective justification.

10. Submission Guidelines

Each group must:

- Submit complete C++ source code files.
- Submit a project report (PDF) briefly explaining class design, data structure choices, and algorithms in IEEE format using LaTeX.
- Give live demo during viva.

Late submissions will not be entertained. Plagiarized code or shared logic will result in **zero marks for all involved members**.

11. Evaluation Criteria

Component	**Marks**
Data Structures Implementation	25
Dataset Handling and Parsing	15
Graph-based Recommendation and Traversal	20
Shortest Path Functionality	15
Code Quality and Documentation	10
Viva & Presentation	15
Total	**100**

12. Guidelines and Rules

1. Each group member must understand the entire project codebase.
2. Clearly comment and explain all recursive functions.
3. Code should compile and run without warnings or errors.
4. The use of external libraries (other than standard I/O and file handling) is prohibited.
5. Evaluation will include both correctness and conceptual understanding.
6. Groups failing to demonstrate their project during viva will be heavily penalized.